

USU-Society Subsystem Overview

Student ID: 19367388

Student Name: Dongsheng Tai

Subsystem: USU-Society

The subsystem I worked on is the USU-Society Subsystem. This subsystem mainly supports society leaders in managing society-related functions in the student union system. In my model, the main user is the Society Leader. They can manage society information, society members, communication with members, society events, event promotion, and society elections.

In my individual part, I focus on the Create Society Event function, because it connects several important parts of the subsystem. When a society leader creates a new event, the system needs to check the leader's authority, validate the event data, save the event information, notify society members, and publish event promotion when needed.

Therefore, this use case is suitable for showing the relationship between the use case model, activity diagram, component design, class diagram, and sequence diagram.

2. Use Case Diagram

The use case diagram shows the main functions that the Society Leader can use in the USU-Society Subsystem. The Society Leader is the main external actor, and all use cases are placed inside the system boundary of the USU-Society Subsystem. This means these functions are within the scope of this subsystem.

The main functions include activating a new society, updating society registration data, requesting society termination, managing society membership, communicating with members, monitoring society operations, creating society events, managing event promotion, updating event information, and running society elections. These use cases cover the main tasks that a society leader may need in daily society management.

Set Up Ticket Registration is modelled as an extend use case of Create Society Event, because not every event needs ticket registration. It only happens when an event requires registration or ticket control. Review Member Feedback is modelled as an extend use case of Monitor Society Operations, because reviewing feedback is an optional activity within society monitoring. Manage Event Promotion includes Publish Event Promotion, which means publishing promotion information is part of the event promotion management function.

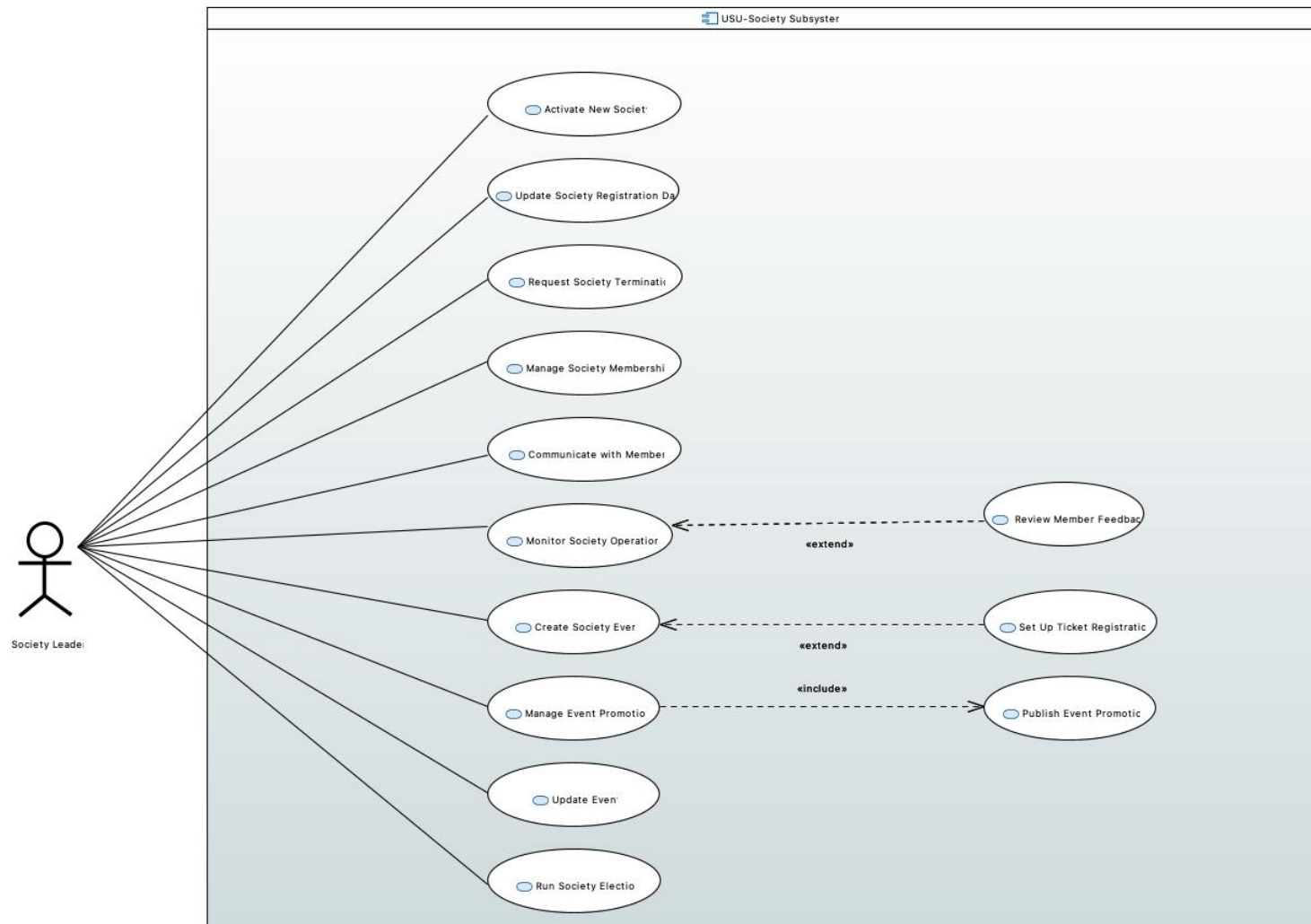


Figure 1. Use Case Diagram for USU-Society Subsystem

3. Activity Diagram for Create Society Event

The activity diagram describes the detailed process of the Create Society Event use case. The diagram uses three main areas: Society Leader, Data / Object Nodes, and USU-Society System. This makes the relationship between user actions, data transfer, and system processing clearer.

The process starts when the Society Leader chooses to create a new event. The system then sends an event creation form, and the leader fills in the event details, such as event name, theme, time, venue, capacity, ticket requirement, and promotion materials. After the form is completed, the system validates the event data. If the data is invalid, the system displays the error message “Complete required fields” and returns the user to the event details form. If the data is valid, the flow continues to the ticket registration check.

For the ticket-related decision, if the event needs ticket registration, the system first performs Set Up Ticket Registration and then saves the event. If ticket registration is not required, the system saves the event directly. After the event is saved, the system notifies society members and runs three parallel actions: Announce Event, Create Society Event Page, and Push Promotions to Members. Finally, the system confirms that the event has been created, returns a success message, and the Society Leader can view the event confirmation.

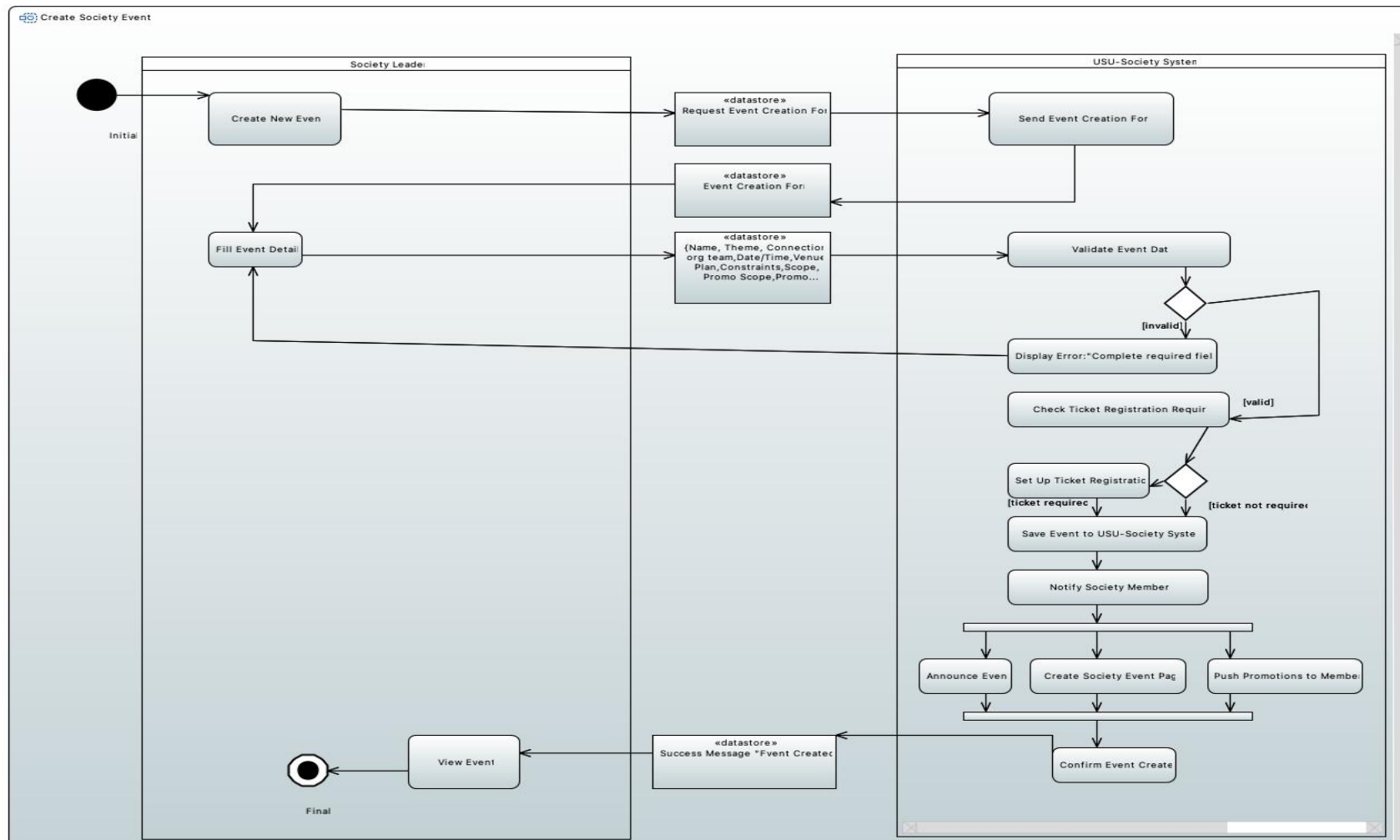


Figure 2. Activity Diagram for Create Society Event

4. Component Diagram for USU–Society Subsystem

The component diagram shows the overall architecture design of the USU-Society Subsystem. This subsystem is divided into a mobile application layer and a backend service layer. The mobile application part allows the Society Leader to access society management functions through the app. The backend part contains several service components, and each component is responsible for a separate functional area.

In the diagram, Society Event Service is the main component related to event management. It provides functions such as creating events, updating events, viewing events, and managing event status through the `ISocietyEventService` interface. To complete the event creation process, this service also depends on other components. For example, Identity Service is used to verify the society leader's authority, Notification Service is used to notify society members, and Event Promotion Service is used to publish event promotion information.

This component structure follows a microservice-style design. Different services are connected through interfaces, instead of putting all functions into one large component. This makes the system structure clearer and also makes future maintenance and extension easier.

4.1 Component Specification

The following table gives more details about the responsibilities of the main components and interfaces in the component diagram. This helps the diagram show not only the structure, but also what each component does in the USU-Society Subsystem.

Component / Interface	Responsibility
USU-SocietyMobileApp	Provides the client-side application for Society Leader to access society management functions.
Session And Access Controller	Handles login and access control before protected functions can be used.
Society Administration Controller	Receives society administration requests from the mobile app.
Membership Control Controller	Receives membership management requests from the client side.
Member Communication Controller	Supports communication functions between Society Leader and society members.
Society Event Controller	Receives event creation and event update requests from the mobile app.

SocietyLeaderGUI	Provides the user interface for viewing notifications and society-related information.
Authentication Service	Checks user identity and leader permission.
Society Administration Service	Handles society administration logic, such as activation, update and termination.
Membership Service	Handles society membership management logic.
Member Communication Service	Provides backend support for member communication.
Society Event Service	Handles event creation, event update, event saving, notification and promotion coordination.
Notification Service	Sends society and event notifications to members.
Authenticate	Interface used by the client side to request authentication.
Manage Society Events	Interface used to connect the event controller with Society Event Service.
ReceiveNotifications	Interface used to return notification-related information to the Society Leader side.
CheckPermission	Required interface for checking whether the current leader has permission.
PushSocietyUpdate	Required interface for pushing society update information.
PushEventPromotion	Required interface for publishing event promotion information.

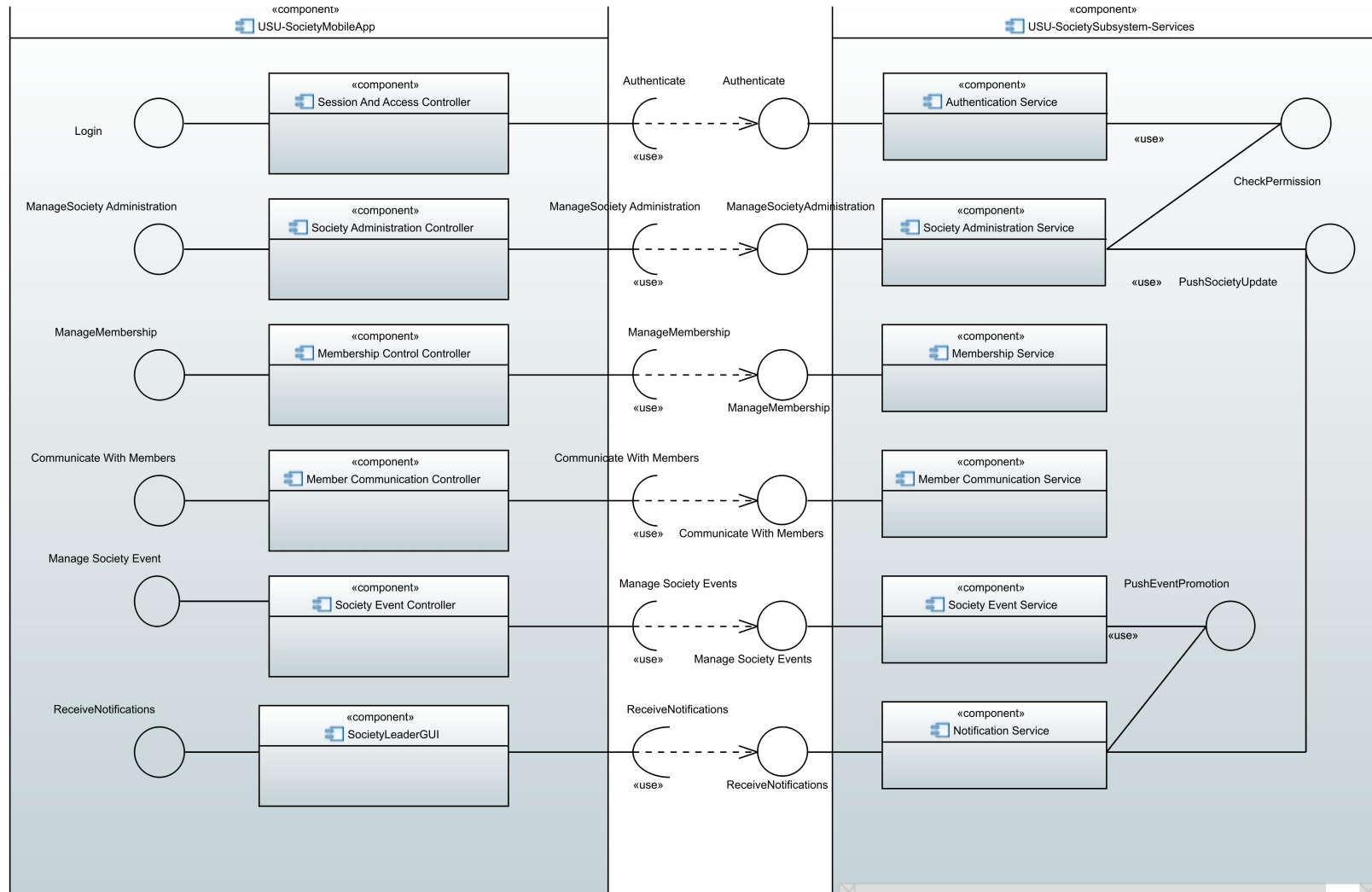


Figure 3. Component Diagram for USU-Society Subsystem

5. Class Diagram for Society Event Service

The class diagram shows the detailed design of the Society Event Service microservice. This microservice is mainly responsible for society event functions, such as creating events, updating events, viewing events, and changing event status. In the diagram, `ISocietyEventService` is the interface provided by this microservice. The `SocietyEventService` class implements this interface and contains the main business operations.

`SocietyEventService` works with several external interfaces and classes. It uses `IdentityService` to check whether the Society Leader has permission to manage the related society. It uses `EventRepository` to save and read Event data. It also uses `NotificationService` to notify society members, and `EventPromotionService` to publish event promotion information. This design allows the event management service to focus on the core business logic, while identity checking, notification, and promotion are handled by separate services.

The diagram also includes domain classes such as `Event`, `Society`, and `SocietyLeader`. The `Event` class stores the main event information, such as title, theme, date, venue, capacity, ticket required, promotion scope, and status. The relationship between `Society` and `Event` shows that one society can have many events. `EventRepository` is also associated with multiple `Event` objects, which means it is responsible for managing event data.

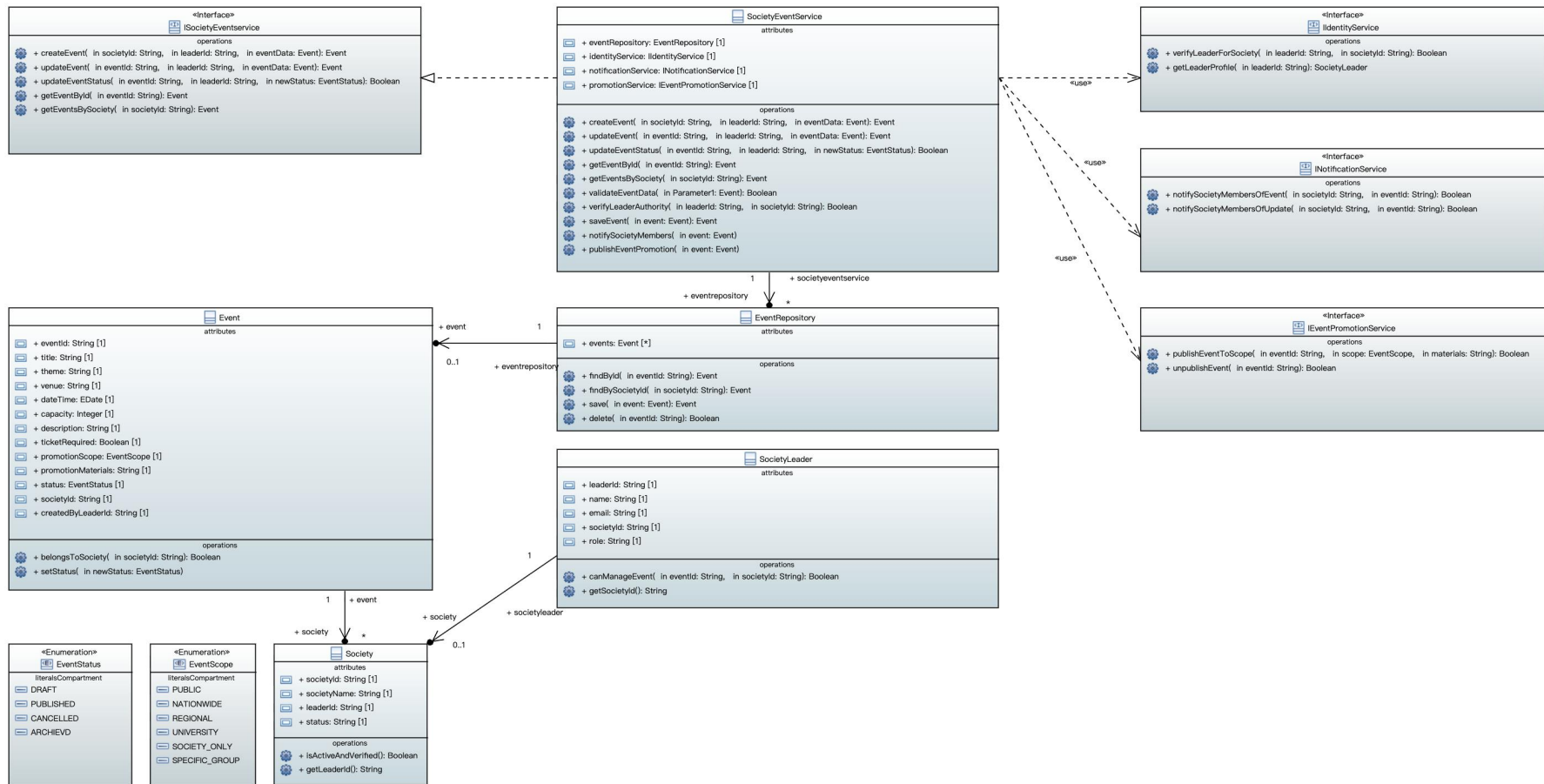


Figure 4. Class Diagram for Society Event Service

6. Sequence Diagram for Create Society Event

The sequence diagram shows how the Create Society Event function is processed between different objects and services. The process starts when the Society Leader sends a createEvent request. The request first goes through the ISocietyEventService interface, and then SocietyEventService handles the main business logic.

Before the event is created, SocietyEventService calls the verifyLeaderForSociety method from IIdentityService. This is used to check whether the leader has permission to create an event for the selected society. If the authorisation check is successful, SocietyEventService continues by calling EventRepository to save the event data. After the event is saved, the system uses INotificationService to notify society members and uses IEventPromotionService to publish event promotion information. Finally, SocietyEventService returns eventCreated, and the interface returns eventCreationConfirmed to the Society Leader.

The diagram also shows failure cases, such as validationFailed and authorisationFailed. This shows that the system does not only handle the normal event creation flow, but also considers invalid data and insufficient authorisation. Overall, the sequence diagram is consistent with the interfaces, methods, and service dependencies shown in the class diagram.

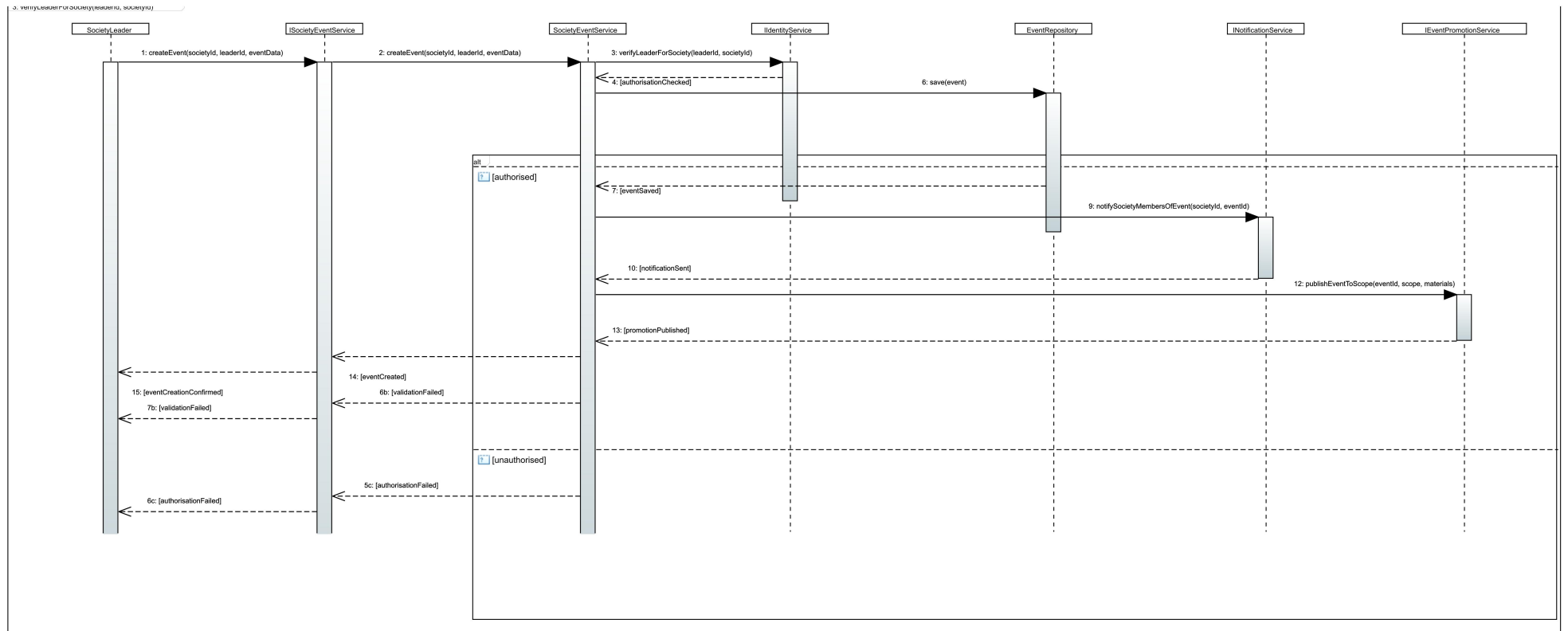


Figure 5. Sequence Diagram for Create Society Event

7. Consistency Between Models

The diagrams follow a consistent design logic. The use case diagram first shows the main functions that the Society Leader can use in the USU-Society Subsystem. Among these functions, Create Society Event is the main use case developed further in the activity diagram, class diagram, and sequence diagram.

The activity diagram describes the detailed business process of Create Society Event, including filling in event information, validating event data, checking whether ticket registration is needed, saving the event, notifying members, and publishing event promotion. These steps also match the main messages in the sequence diagram, such as `createEvent`, `save(event)`, `notifySocietyMembersOfEvent`, and `publishEventToScope`.

The component diagram explains how these functions are supported by different service components, such as Society Event Service, Identity Service, Notification Service, and Event Promotion Service. The class diagram then gives a more detailed design of `SocietyEventService`, showing how it uses `IIdentityService`, `EventRepository`, `INotificationService`, and `IEventPromotionService` to complete the event creation process.

The sequence diagram finally shows the interaction between these classes and interfaces in time order. Therefore, the five diagrams are not separate from each other. They describe the same subsystem from different levels: requirements, workflow, architecture, detailed structure, and object interaction.

8. Unit Test Plan

The unit test plan focuses on the main operations of SocietyEventService. Since this coursework is based on modelling and design rather than implementation, the following tests describe how the methods should be tested when the service is implemented. The tests cover validation, authorisation, event saving, notification and promotion, which are also the main behaviours shown in the class diagram and sequence diagram.

Test ID	Method / Unit Under Test	Test Aim	Test Data / Input	Expected Result
UT1	validateEventData(eventData)	Check that complete event data is accepted.	eventData = {title: "Welcome Party", theme: "Social", dateTime: "2026-05-10 18:00", venue: "Main Hall", capacity: 80, ticketRequired: false}	The method returns true. The event data can continue to the ticket checking stage.
UT2	validateEventData(eventData)	Check that missing required fields are rejected.	eventData = {title: "", theme: "Social", dateTime: "2026-05-10 18:00", venue: "Main Hall", capacity: 80}	The method returns false. The event should not be saved and an error message should be produced.
UT3	validateEventData(eventData)	Check that invalid capacity is rejected.	eventData = {title: "Music Night", dateTime: "2026-05-12 19:00", venue: "Union Room", capacity: -5}	The method returns false because capacity cannot be negative.
UT4	verifyLeaderAuthority(leaderId, societyId)	Check that an authorised society leader is accepted.	leaderId = "L001", societyId = "SOC101", where L001 is the leader of SOC101.	The method returns true, so the create event process can continue.

UT5	verifyLeaderAuthority(leaderId, societyId)	Check that an unauthorised leader is rejected.	leaderId = "L009", societyId = "SOC101", where L009 does not manage this society.	The method returns false. The service should return authorisationFailed.
UT6	createEvent(societyId, leaderId, eventData)	Check successful event creation with valid input.	Valid societyId, valid leaderId, and complete eventData for a society event.	A new Event object is created and returned with a generated eventId.
UT7	saveEvent(event) / EventRepository.save(event)	Check that the event is stored correctly.	event = {eventId: "E001", title: "Welcome Party", societyId: "SOC101"}	The event is saved in EventRepository, and it can be found later by findById("E001").
UT8	notifySocietyMembers(event)	Check that members are notified after event creation.	Saved event with eventId = "E001" and societyId = "SOC101".	A notification request is sent through INotificationService.notifySocietyMembersOfEvent(societyId, eventId).
UT9	publishEventPromotion(event)	Check that promotion is published when promotion data is provided.	Event with promotionScope = "UNIVERSITY" and promotionMaterials = "Poster and short text".	A promotion request is sent through IEventPromotionService.publishEventToScope(eventId, scope, materials).
UT10	publishEventPromotion(event)	Check that promotion is skipped or limited when no promotion material is provided.	Event with promotionScope = "SOCIETY_ONLY" and empty promotionMaterials.	No wider promotion is published, or the event is only visible to society members.
UT11	updateEvent(eventId, leaderId, eventData)	Check that an authorised society leader can update	eventId = "E001", leaderId = "L001", updated eventData with venue = "Room B" and capacity = 100.	The event details are updated and saved in EventRepository.

		an existing event.		
UT12	updateEventStatus(eventId, leaderId, newStatus)	Check that an authorised society leader can change the event status.	eventId = "E001", leaderId = "L001", newStatus = PUBLISHED.	The event status is changed to PUBLISHED, and the updated event can be retrieved.
UT13	getEventById(eventId)	Check that an existing event can be retrieved by its id.	eventId = "E001", where the event already exists in the repository.	The correct Event object is returned with matching eventId, title and societyId.
UT14	getEventsBySociety(societyId)	Check that all events for a selected society can be listed.	societyId = "SOC101", where the society has several events.	A list of events belonging to SOC101 is returned, and events from other societies are not included.

These unit tests are selected based on the main methods in the SocietyEventService class diagram. They cover event validation, leader authorisation, event creation, event update, event status update, event retrieval, repository saving, member notification and event promotion. They also match the main messages and services shown in the sequence diagram. The aim is to check the correctness of each small unit before testing the whole Create Society Event workflow at system level.

9. System Test Plan

The system test plan focuses on the complete Create Society Event use case from the Society Leader's point of view. These tests are based on the activity diagram and sequence diagram. They cover the normal successful flow and the main alternative flows, including invalid event data, authorisation failure, ticket registration and event promotion.

Test ID	Scenario	Preconditions	Test Steps	Expected Result
ST1	Create a normal society event without ticket registration.	Society Leader is logged in and authorised for SOC101.	1. Select Create New Event. 2. Fill in title, theme, date/time, venue and capacity. 3. Select ticketRequired = false. 4. Submit the form.	The system validates the data, saves the event, notifies society members, and shows Success Message: Event Created.
ST2	Create a society event with ticket registration.	Society Leader is authorised for the society.	1. Select Create New Event. 2. Fill in valid event details. 3. Select ticketRequired = true. 4. Complete ticket registration setup. 5. Submit the form.	The system sets up ticket registration, saves the event, notifies members, and shows event creation confirmation.
ST3	Submit the form with missing required fields.	Society Leader has opened the event creation form.	1. Leave title or dateTime empty. 2. Submit the form.	The system detects invalid data and displays Complete required fields. The user is returned to Fill Event Details.
ST4	Unauthorised user tries to create an event.	User is not the leader of the	1. User tries to create an event for SOC101. 2. System checks leader	The system rejects the request and returns authorisationFailed. The event is not saved.

		selected society.	authority.	
ST5	Create event with university-wide promotion.	Society Leader is authorised and event data is valid.	1. Fill in event details. 2. Set promotionScope = UNIVERSITY. 3. Add promotion materials. 4. Submit the form.	The system saves the event, notifies members, and sends promotion data to IEventPromotionService.
ST6	Create event without promotion material.	Society Leader is authorised and event data is valid.	1. Fill in valid event details. 2. Leave promotion materials empty or choose society-only scope. 3. Submit the form.	The event is created successfully. Wider promotion is not published, but the event can still be viewed by the society members.
ST7	Notify society members after event is created.	Event has been saved successfully.	1. Complete a valid event creation process. 2. Observe the notification step.	INotificationService sends a notification to society members about the new event.
ST8	View event confirmation after successful creation.	Event creation has completed successfully.	1. Wait for the success message. 2. Select or view the created event confirmation.	The Society Leader can see the event confirmation and the created event details.
ST9	Event repository failure during saving.	Event data is valid, but the repository is unavailable or returns an error.	1. Submit a valid event creation form. 2. Simulate repository save failure.	The event is not confirmed as created. The system should return a save failure message or keep the event creation process from showing success.
ST10	Boundary test for event capacity.	Society Leader is creating an event.	1. Submit an event with capacity = 1. 2. Submit another event with a very large capacity, such as capacity = 10000.	Capacity 1 should be accepted if the venue allows it. Very large capacity should be rejected or flagged if it is not realistic for the venue.

The system test cases are designed to cover the main paths in the activity diagram. ST1 and ST2 test the normal successful paths with and without ticket registration. ST3 and ST4 test the two important failure paths: invalid event data and failed authorisation. ST5 and ST6 test the promotion-related behaviour, which is also represented in the component, class and sequence diagrams. ST7 and ST8 check the final notification and confirmation behaviour. ST9 and ST10 add more robustness by considering repository failure and boundary values for capacity. Together, these tests provide stronger coverage of the Create Society Event workflow